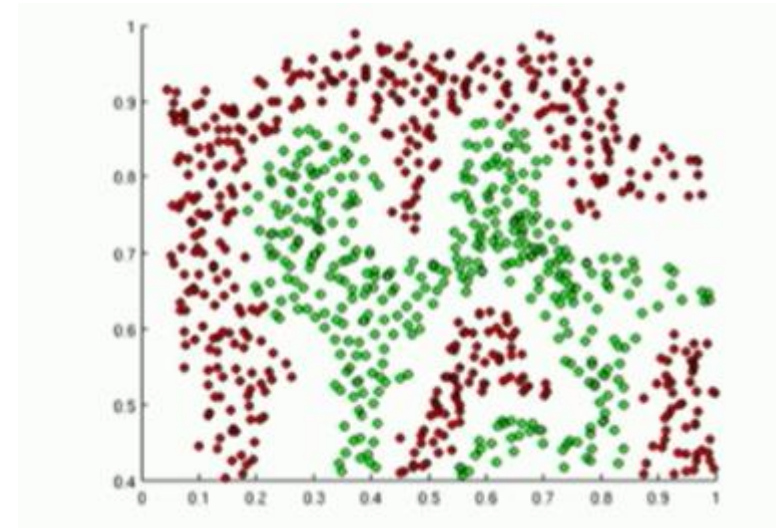


Importance of activation functions

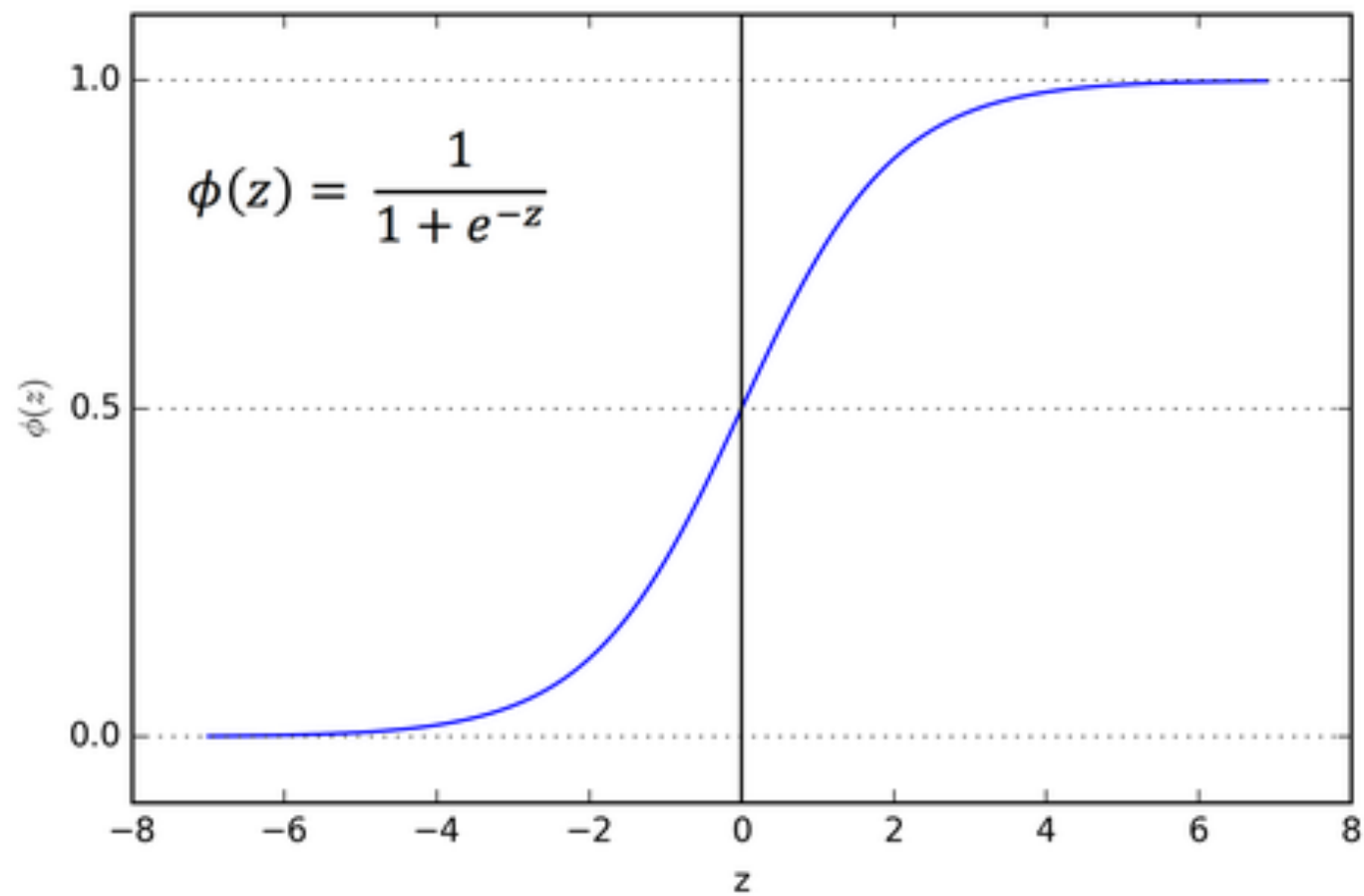
- The purpose of activation functions is to introduce non-linearities in the network



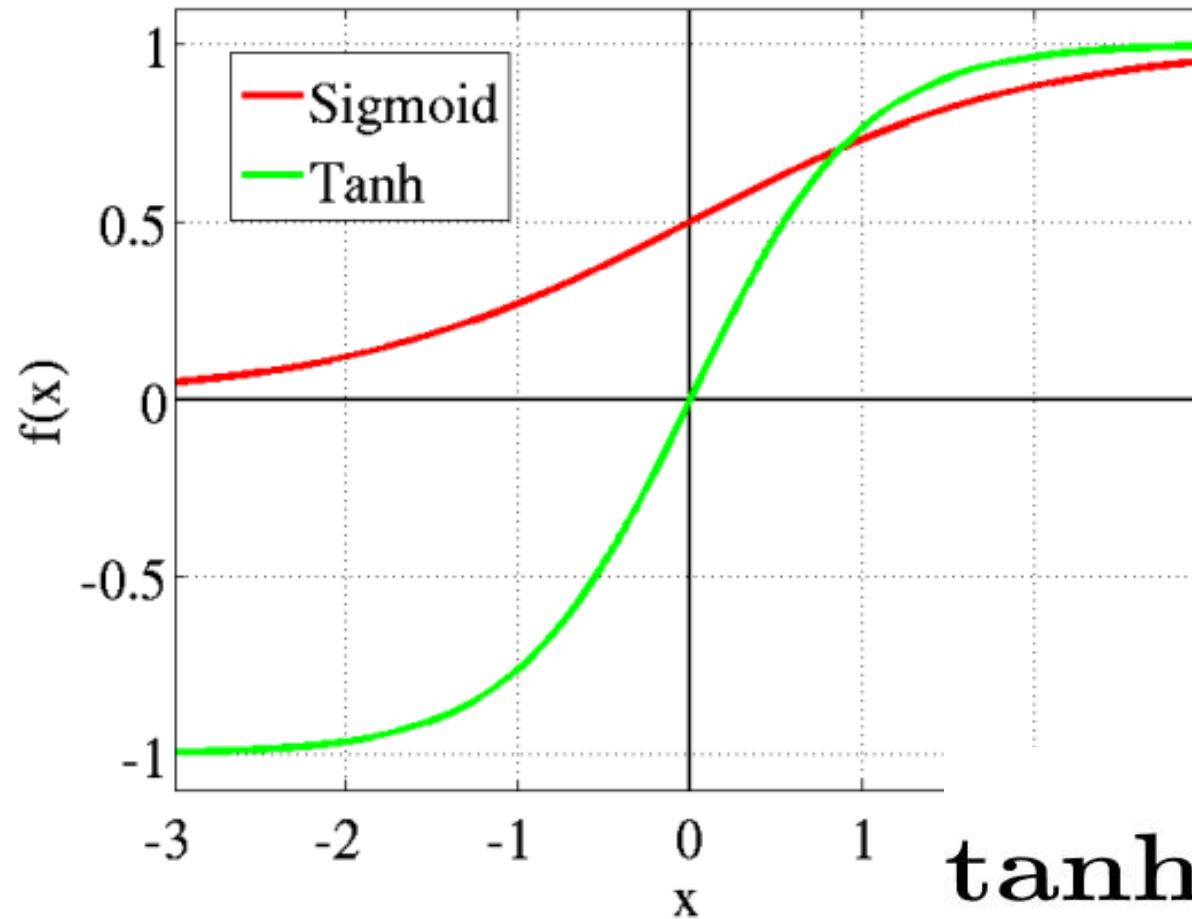
Importance of activation functions

- Neural Network without activation function:
- Sum of dot products between inputs and weights.
- **Input: x_0, x_1, x_2 then Output = $w_0x_0 + w_1x_1 + w_2x_2$**
- Just a linear output – no matter how many such neurons you stack up!
- Real world data is almost always Non-Linear
- Activation Function adds Non linearity to the Neural Network
- Characteristics of good activation function:
- Non-linearity
- Differentiable

Sigmoid

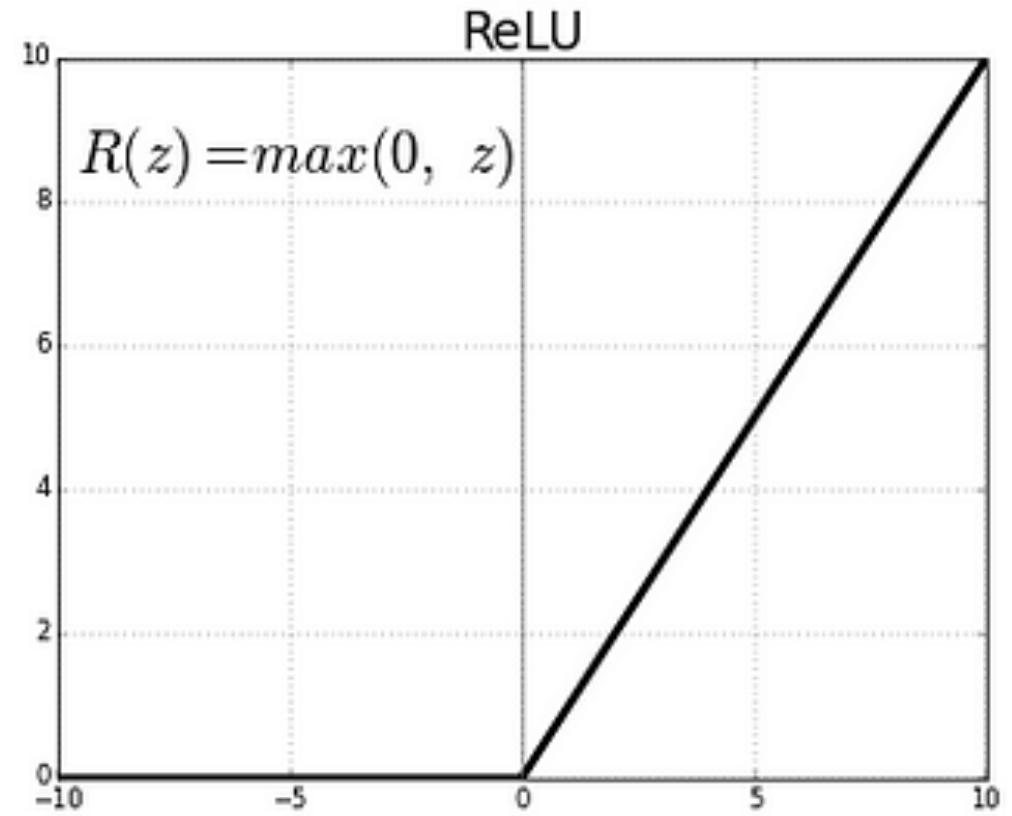
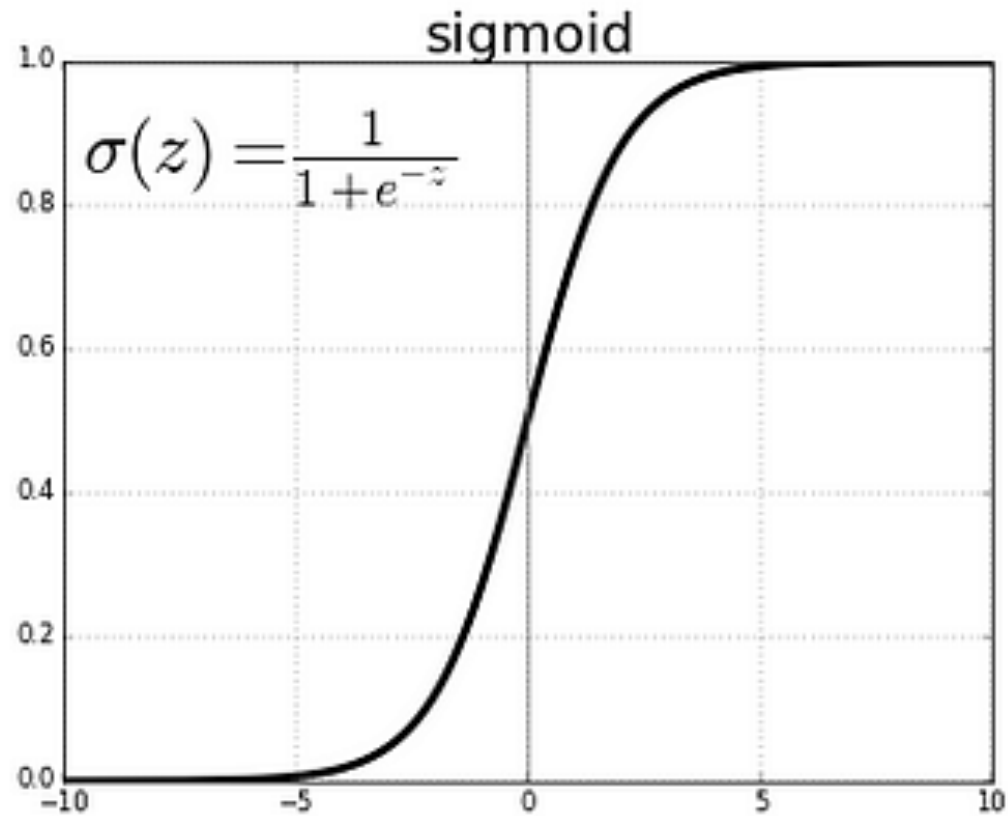


Tanh or hyperbolic tangent

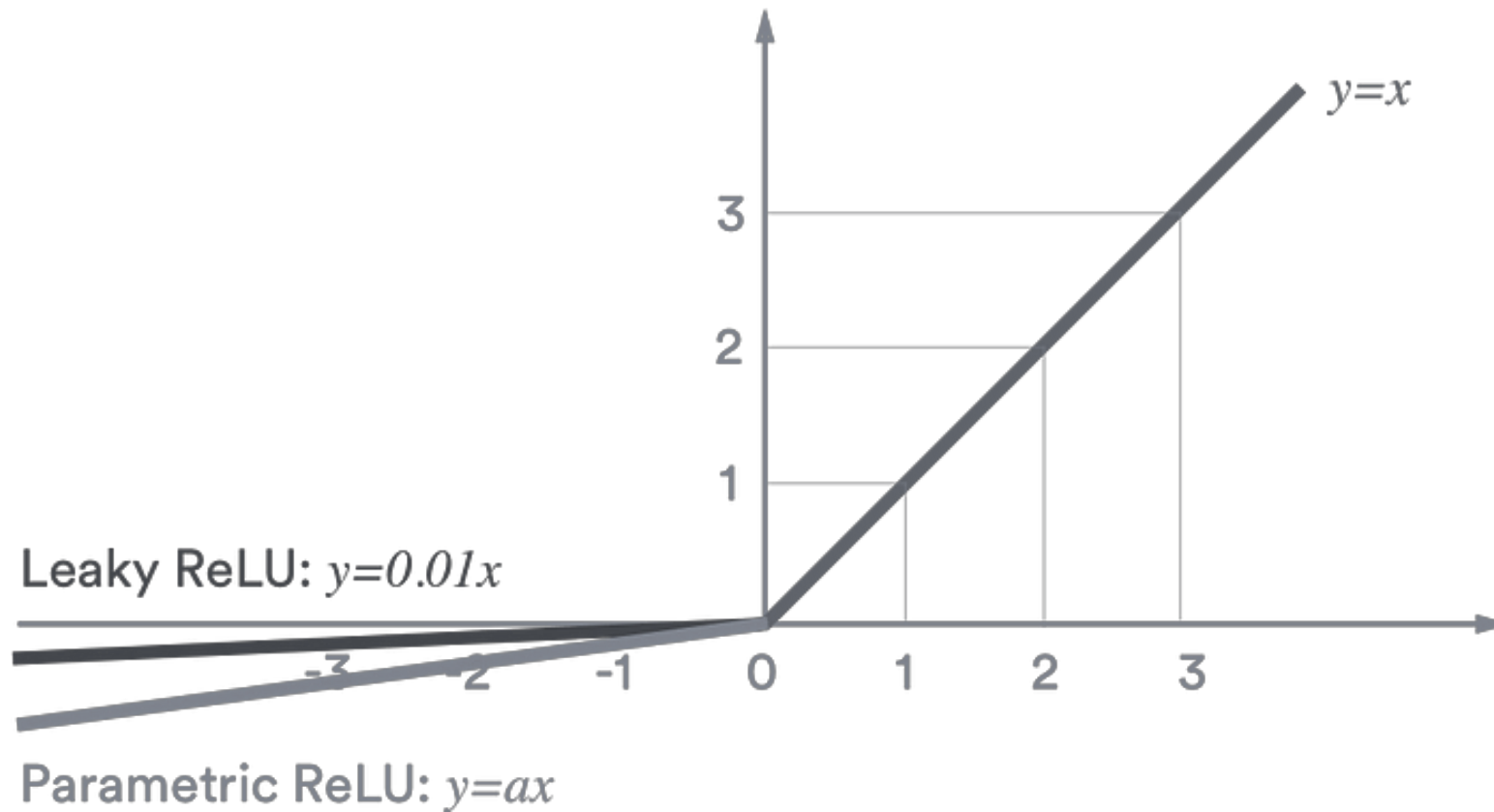


$$\tanh x = \frac{e^x - e^{-x}}{e^x + e^{-x}}$$

ReLU (Rectified Linear Unit)



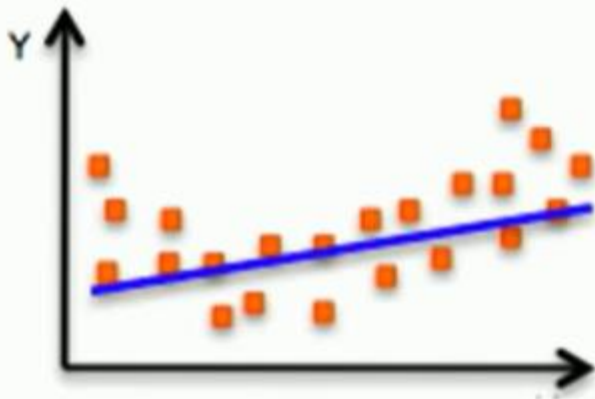
Leaky ReLu / Parametric ReLU



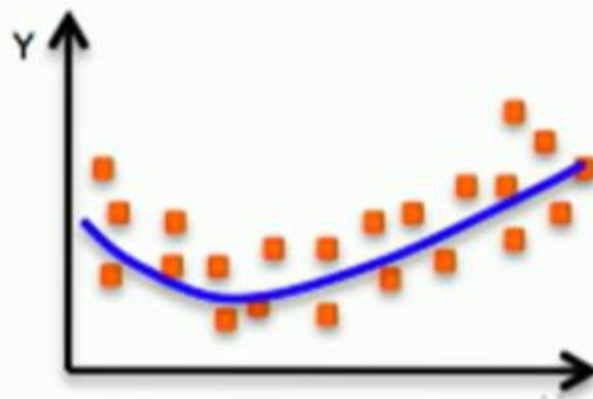
Softmax

$$\sigma(\vec{z})_i = \frac{e^{z_i}}{\sum_{j=1}^K e^{z_j}}$$

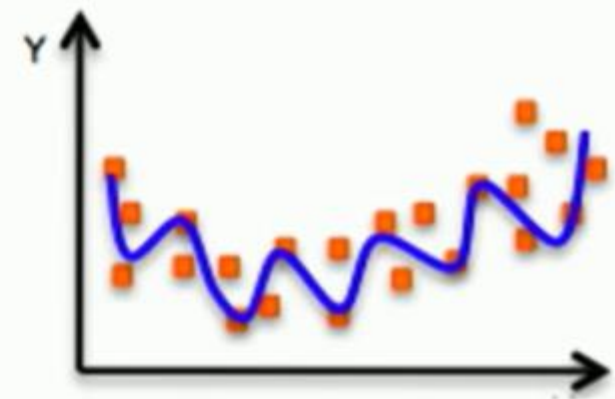
The problem of overfitting



Underfitting
Model does not have capacity
to fully learn the data



← Ideal fit →



Overfitting
Too complex, extra parameters,
does not generalize well

Regularization

- **What is it?**
 - Technique that constraints our optimization problem to discourage overly complex models
 - Any modification or change in the learning algorithm that helps reduce its error over unseen data (generalization error).
- **Why do we need it?**
 - To improve the generalization of our model on unseen data

Parameter Norm Penalties

- The capacity of the models like neural networks, linear or logistic regression is limited by adding a parameter norm penalty $\Omega(w)$ to the objective function.

$$\hat{\mathcal{L}}(W) = \mathcal{L}(W) + \alpha\Omega$$

- α is regularization rate
- $\alpha = 0$ results in no regularization, while larger values imply greater regularization

L2 Regularization

- *Regularization for **simplicity**.*
- Simplest and most common form of regularization
- Also known as weight decay or ridge regression
- If we take the model complexity as a function of weights, the complexity of a feature is proportional to the absolute value of its weight.
- It acts like a force that removes a small percentage of weights at each iteration.
- ***L2 regularization penalizes $(weight)^2$***

L2 Regularization

- The regularization term

$$\Omega(W) = ||W||_2^2 = \sum_i \sum_j w_{ij}^2$$

- New loss function

$$\hat{\mathcal{L}}(W) = \frac{\alpha}{2} ||W||_2^2 + \mathcal{L}(W) = \frac{\alpha}{2} \underbrace{\sum_i \sum_j w_{ij}^2} + \mathcal{L}(W)$$

L2 Regularization

- The gradient of the new loss function

$$\nabla_W \hat{\mathcal{L}}(W) = \alpha W + \nabla_W \mathcal{L}(W)$$

- Weight update rule

$$W_{new} = W_{old} - \epsilon(\alpha W_{old} + \nabla_W \mathcal{L}(W_{old}))$$

- Same as

$$W_{new} = (1 - \epsilon\alpha)W_{old} - \epsilon\nabla_W \mathcal{L}(W_{old})$$

L1 Regularization

- *Regularization for **sparsity***
- Also known as Lasso regression
- It is beneficial in handling sparse vectors which consist of mostly zeroes. Sparse vectors typically result in very high-dimensional feature vector space, making the model very difficult to tune.
- L1 regularization forces the weights of uninformative features to be zero by subtracting a small amount from the weight at each iteration and thus making the weight zero, eventually.
- ***L1 regularization penalizes $|weight|$.***

L1 Regularization

- We simply use a different regularization term Ω , which is the sum of the absolute values of the weight parameters in a weight matrix:

$$\Omega(W) = ||W||_1 = \sum_i \sum_j |w_{ij}|$$

- New loss function:

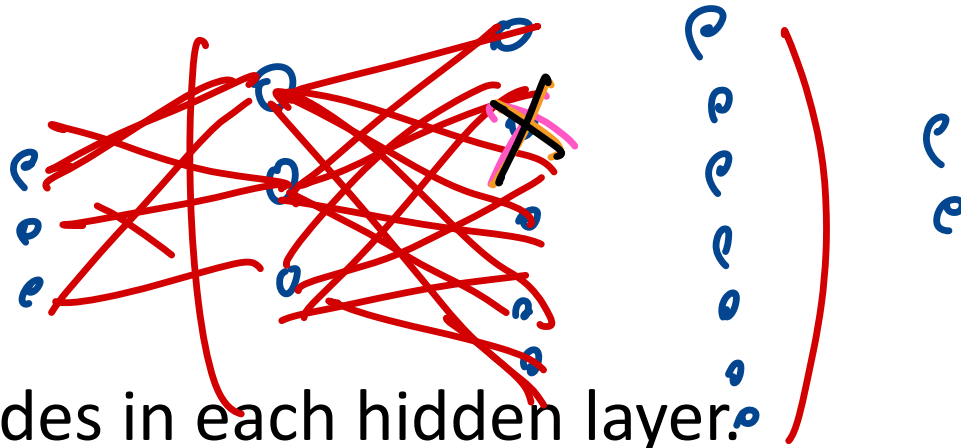
$$\hat{\mathcal{L}}(W) = \alpha ||W||_1 + \mathcal{L}(W)$$

L1 Regularization

- It gives a sparse solution
- Used for feature selection
- The derivative of the new loss function: the sum of the gradient of the old loss function and sign of a weight value times alpha

$$\nabla_W \hat{\mathcal{L}}(W) = \alpha \text{sign}(W) + \nabla_W \mathcal{L}(W)$$

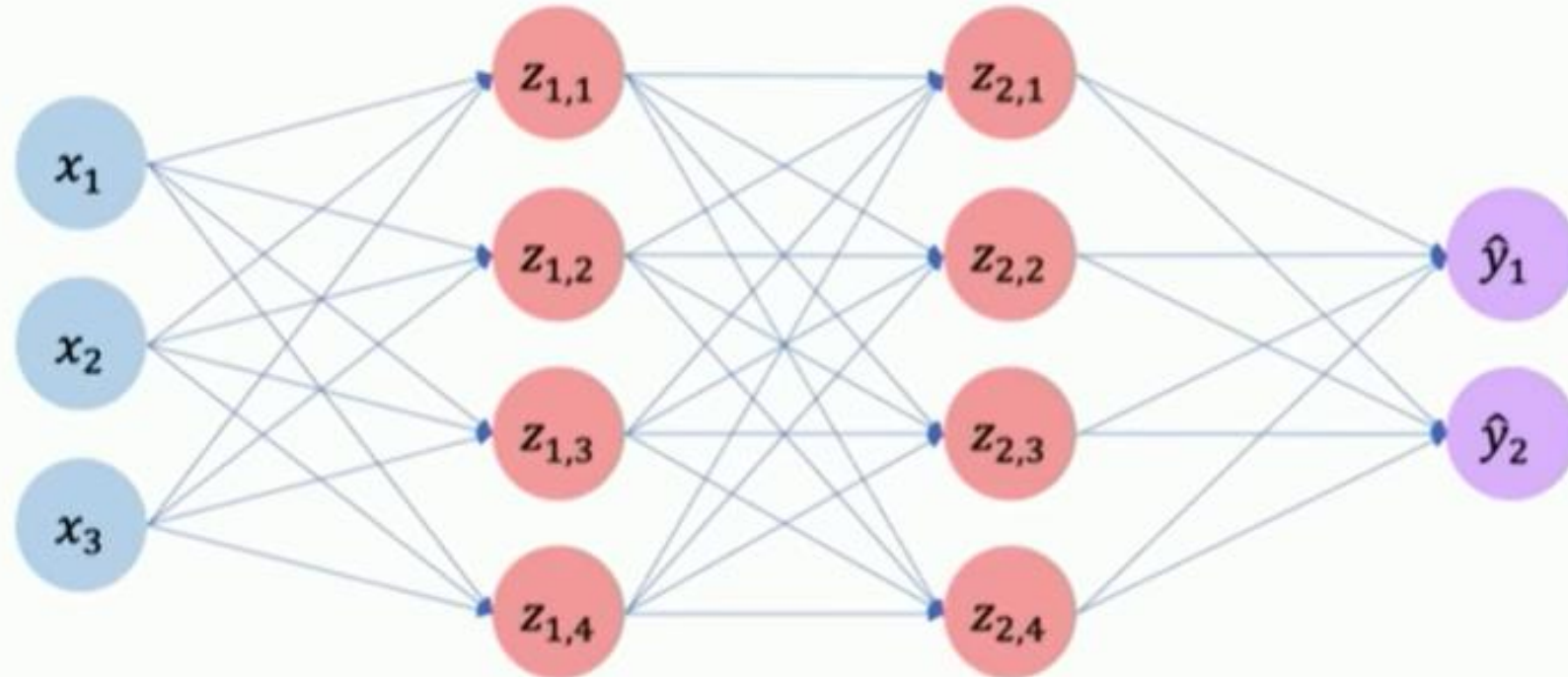
Dropout



- While training, eliminate nodes in each hidden layer.
- For each hidden layer and for each node in the hidden layers, a probability of keeping the node is set.
- For each example, nodes are eliminated based on the set probability.
- E.g.: If 50 hidden units are there and the “keep probability” is 0.8, then 10 units are dropped out.
- 10 units are dropped out while training with each example.

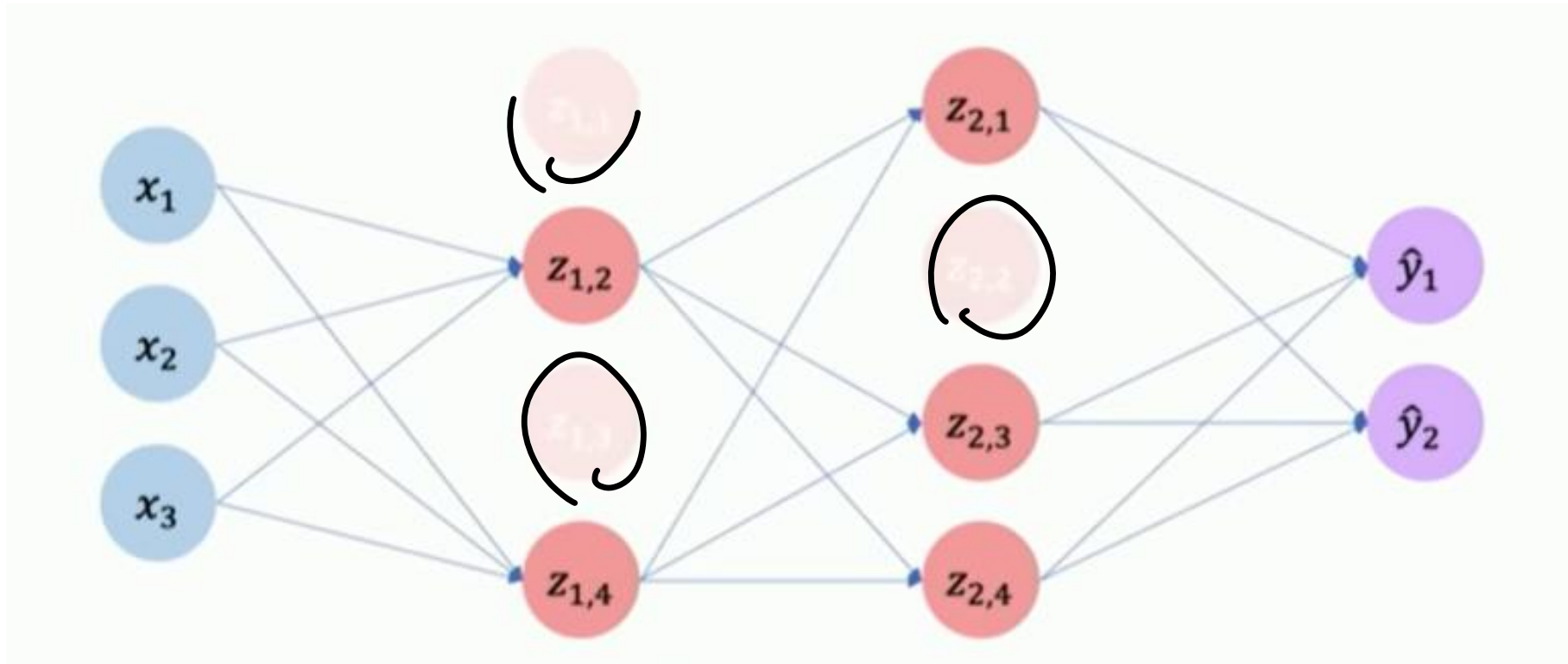
Dropout

- During training, randomly set some activations to 0



Dropout

- Forces the network to not rely on any one node



Early stopping

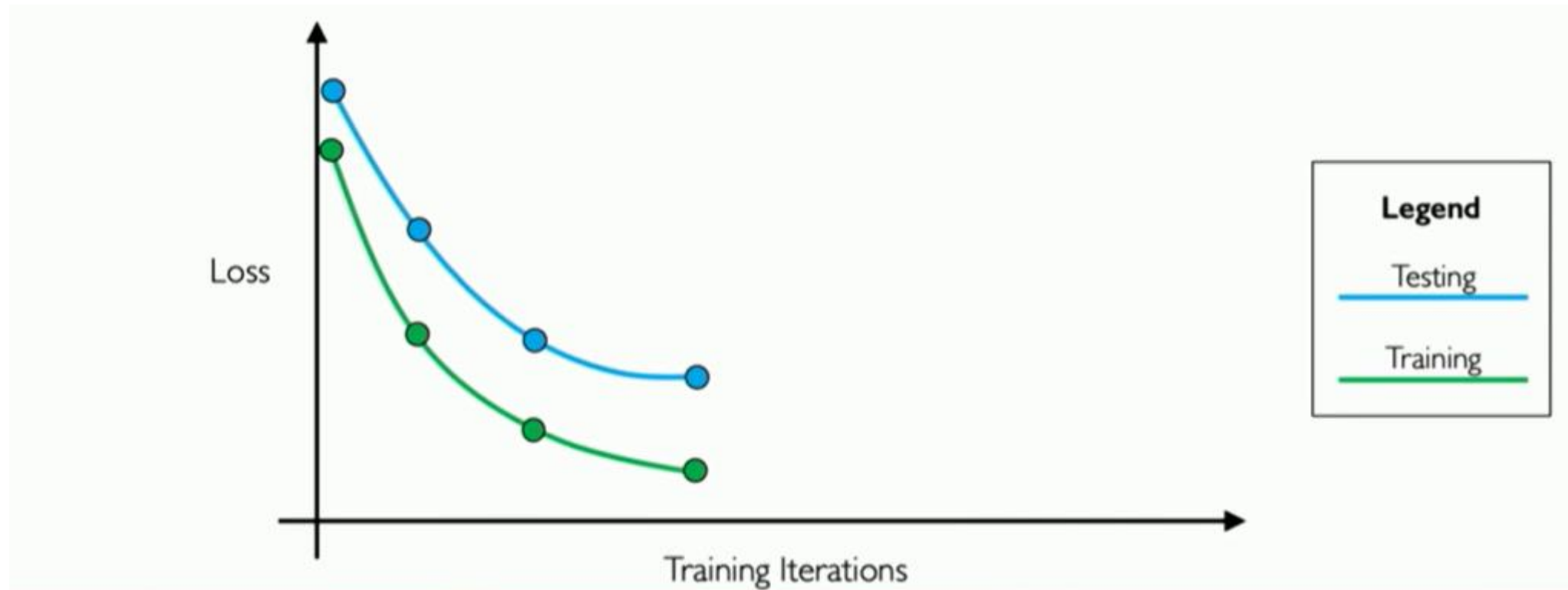
- Stop training before we have a chance to overfit



Early stopping

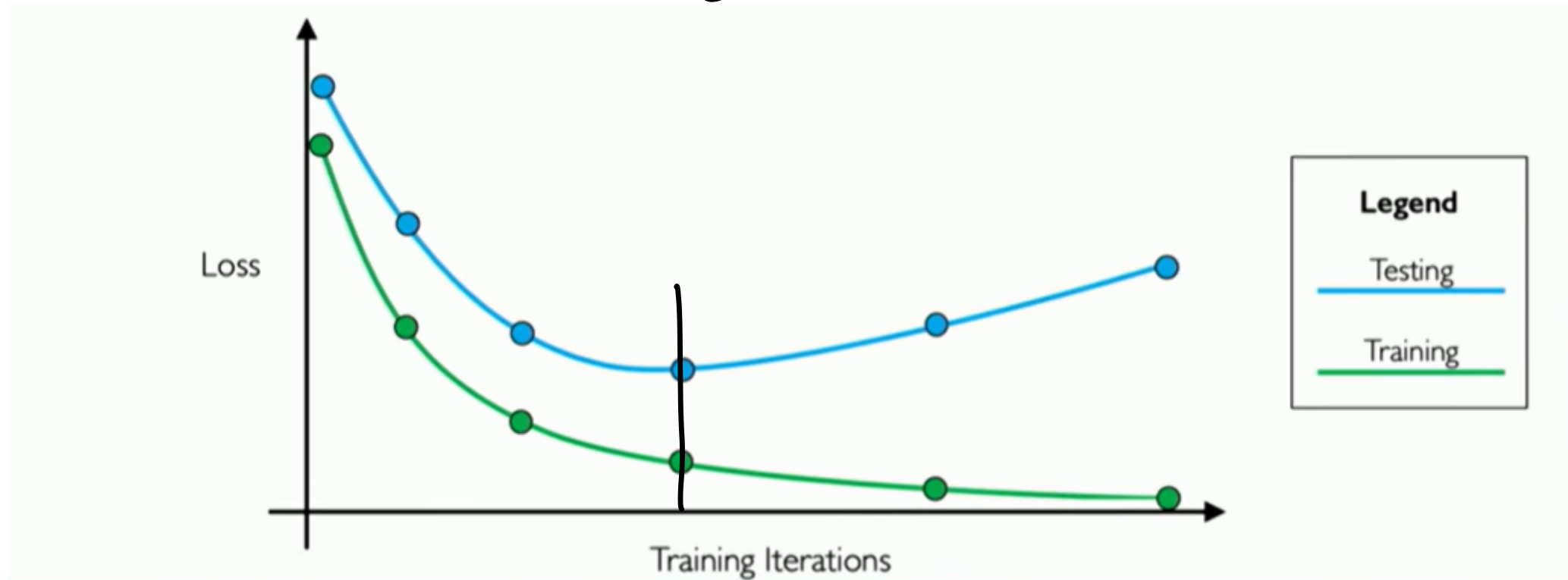


Early stopping



Early stopping

- ① cost func
- ② algo (dropout)
- ③ testing error



Early stopping

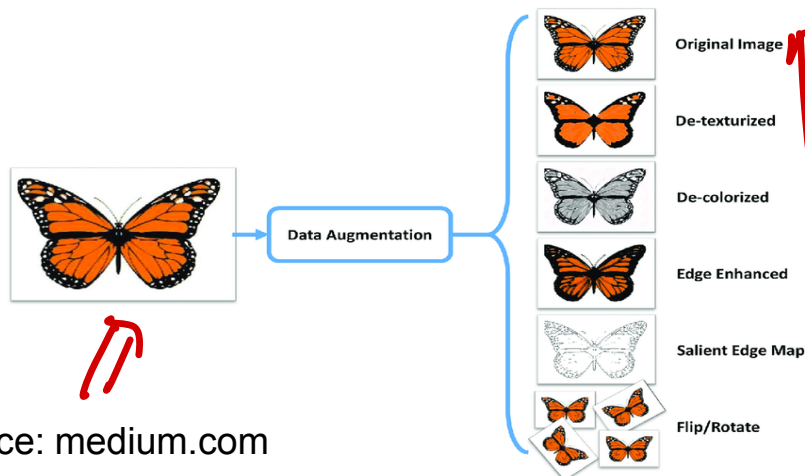


Early stopping

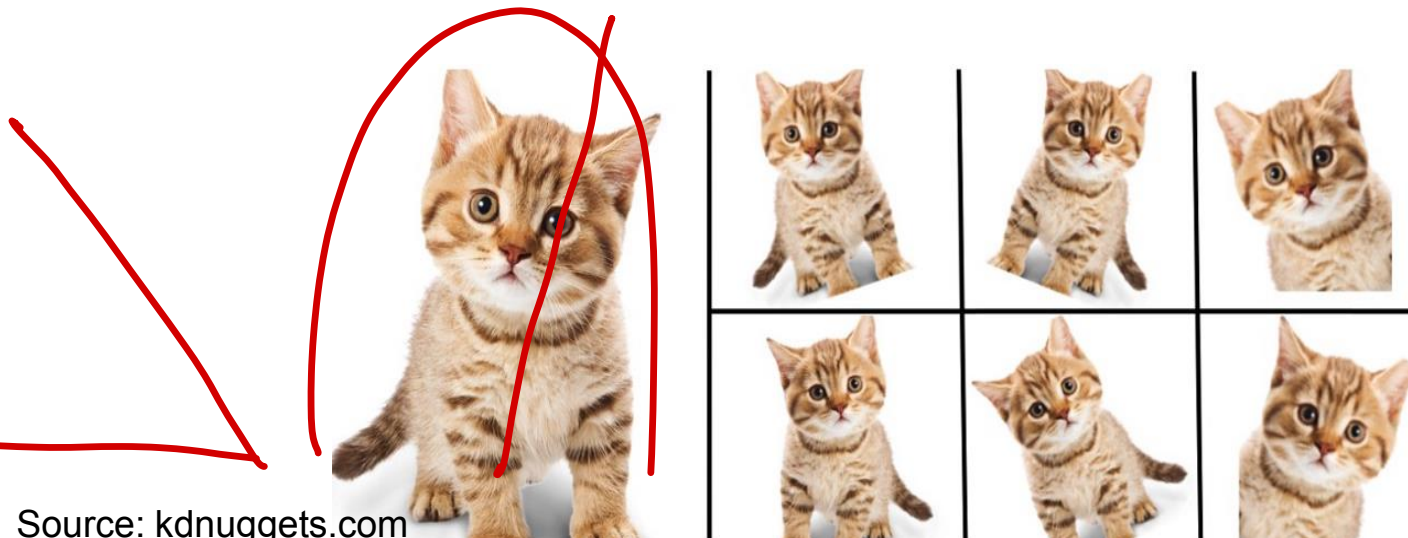


Data augmentation

- The best way to make a machine learning model generalize better is to train it on ***more data***.
- In practice, the amount of data we have is limited.
- One way to get around this problem is to create **fake data** and add it to the **training set**.
- For some machine learning tasks, it is reasonably straightforward to **create new fake data**.



Source: medium.com



Noise injection

- Injecting noise in the input to a neural network can also be seen as a form of data augmentation.
- For many classification and even some regression tasks, the task should still be possible to solve even if small random noise is added to the input.
- Neural networks prove not to be very robust to noise, however.
- One way to improve the robustness of neural networks is simply to train them with random noise applied to their inputs.
- Noise can be applied to hidden units as well. “Dropout” technique is the main development of this approach.

Thank you!

Content adapted from MIT's Intro to Deep Learning course.